

When Is It Safe to Stop Looking at the Image?

A measured threshold in vision language models, and what visual token pruning gets wrong

P. S. Kedar

Supervised by Prof. Divya Saxena | Mentored by Aditya Sharma, PhD Scholar

School of AI and Data Engineering, IIT Jodhpur | Phase 3 Report, September 2026

Subject paper: Balanced Token Pruning (NeurIPS 2025, arXiv:2505.22038)

1. Summary

Vision language models are expensive to run because every image becomes thousands of tokens. Pruning methods throw most of those tokens away as the model runs. Balanced Token Pruning (BTP) is one such method, published at NeurIPS 2025, and its released code also deletes every remaining image token at a fixed layer to save more compute.

We set out to explain why that code destroys the model's ability to read text inside an image. The answer turned out to rest on a quantity nobody measures.

Three findings

1. There is a threshold. Both models we tested keep consulting image tokens until roughly three quarters of the way through the decoder, then release them abruptly. Delete the tokens before that depth and text reading collapses; delete them after and it costs nothing. The threshold is sharp and differs by architecture: 75 per cent of the way through InternVL2-2B, 84 per cent through Qwen2.5-VL.
2. The released code sits one layer on the wrong side of it. BTP deletes at layer 23 of Qwen's 28. Layer 23 retains 27 per cent of baseline reading accuracy; layer 24 retains 75 per cent. One layer is worth 47.7 points.
3. Nobody would notice. With the fault present, the code still scores 96.1 per cent of baseline on the paper's own benchmark suite, inside the 96 to 98 per cent it claims. None of those six benchmarks asks the model to read.

2. The Problem

Earlier phases reproduced the published results on three models within about a point. We then tested on text-heavy benchmarks, which the paper does not report.

- LLaVA-1.5-7B behaved sensibly. TextVQA fell from 46.1 to 40.0, a loss of 6.1 points.
- Qwen2.5-VL-7B collapsed. TextVQA fell from 86.2 to 23.3, a loss of 62.9 points. DocVQA fell from 94.7 to 19.2, ChartQA from 76.8 to 29.0.
- AI2D, a diagram benchmark that rarely needs text read word by word, barely moved at 86.4 to 79.6. It served as our control.

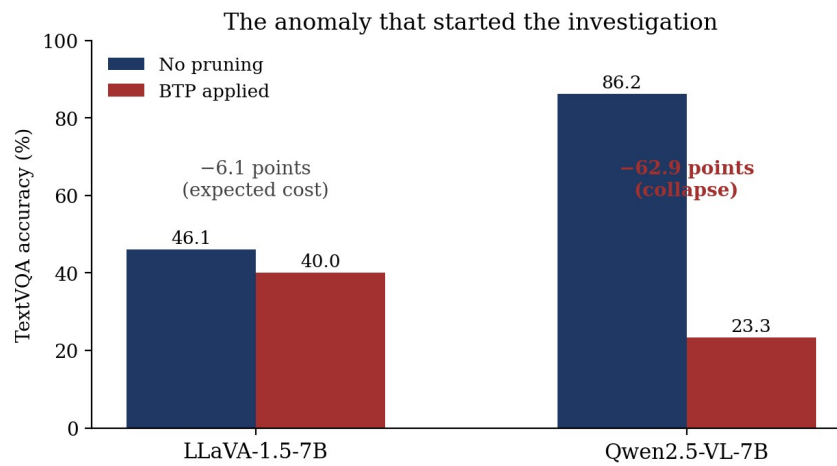


Figure 1. The same method on two model families, two very different outcomes.

3. What We Ruled Out

We listed the candidate causes first and built a test capable of killing each, so that no explanation survived merely because nobody had checked it. Four experiments returned negative before the real cause appeared.

What we tested	What we found	Verdict
Is the selection rule at fault? Ablated it at a fixed budget: BTP, diversity only, attention only, random	All four scored 17.3 to 19.4, inside the error bars. Random did as well as BTP	Not the selector
Are text patches near-duplicates in feature space, as we had assumed?	Text-to-text similarity 0.379, background-to-background 0.438. Text is the more varied region	Our own premise was wrong
Is it a matter of how many tokens are removed?	Removing 10 per cent already gave the full collapse; removing 77 per cent more changed nothing	Not the amount
Would forcing every text token to survive help? Used ground-truth OCR as an oracle	0.197 for the oracle against 0.197 for the control	Not even a perfect selector

Table 1. The four negative results, in the order they were run.

Why all four failed the same way

Each experiment changed something before layer 23 and measured an answer produced after it. Since every image token is deleted at that layer whatever you do, all four were measuring the same floor.

4. The Cause, and the Proof

The clue came from a control we added almost as an afterthought: a setting at 100 per cent retention, where the pruning stages keep every token and should behave exactly like no pruning at all. It scored 0.178, the same as every other budget. We verified the retention value was genuinely reaching the worker process. Something downstream of the budget was destroying the image.

Searching the model file for every branch that touches the hidden-state sequence turned up one candidate.

```

elif layer_index == self.end_layer:      # self.end_layer = 23
    remain_sys_index = torch.tensor(list(range(self.img_start_idx)))
    remain_text_index = torch.tensor(list(range(img_end, total_length)))

    # there is no remain_img_index in the line below
    all_remained_index = torch.cat((remain_sys_index, remain_text_index))
    hidden_states = hidden_states[:, all_remained_index, :]

```

The concatenation joins system tokens and text tokens. There is no `remain_img_index`. Every image token that survived three stages of careful pruning is deleted at layer 23, always. Just above it sits a commented-out version that would have kept some by attention score.

To turn an explanation into a proof we changed one integer, so the branch can never fire, and left everything else alone including the full 12.5 per cent pruning.

Arm	TextVQA	AI2D (control)
No pruning (baseline)	0.8565	0.8750
BTP as released	0.1725	0.8150
BTP with the layer-23 deletion switched off	0.7860	0.8100

Table 2. Job 416735, 200 samples. One line changed, accuracy moves from 0.1725 to 0.7860.

- An intervention, not an observation. One variable changed, everything else fixed.
- The flat curve beforehand was the giveaway: 100, 50 and 12.5 per cent retention all scored 0.178, because nothing survived layer 23 in any of them.
- AI2D hardly moves across all three arms, so this is not a change that simply lifts every score.

5. What the Paper Specifies

Deleting visual tokens at depth is a legitimate design choice, so calling it a fault without checking would be careless. Appendix 7.3 of the paper covers the final pruning stage for each model family.

For LLaVA-v1.5-7B, LLaVA-v1.5-13B, and LLaVA-v1.6-7B, we divide the pruning process into five stages ... In the final stage, all tokens are discarded to maximize inference speed. For Qwen2.5-VL, since its image token processing can be clearly divided into two stages, we retain 25% of the tokens in the fourth stage and 12.5% in the final stage to preserve model performance.

- Discarding everything at the end is a real part of BTP, and the stated reason is speed.
- It is specified for the LLaVA family only.
- For Qwen2.5-VL the paper asks for the opposite: keep 12.5 per cent, and the stated reason is, in the authors' own words, to preserve model performance.
- The released code applies the LLaVA schedule to Qwen anyway, with no check for which model is running.

Two obvious objections do not hold. This is not the VTW baseline leaking in, since the code keeps VTW separately with its own flag and layer number. And it is not implied by the method section, which defines pruning at exactly three layers, each keeping a subset.

6. The Threshold

Knowing that layer 23 was fatal raised a better question. Is depth itself the problem, or is 23 simply the wrong depth? So we swept it. All visual tokens are deleted at a given layer, graded pruning is switched off, and nothing else varies. We did this on Qwen2.5-VL-7B and again on InternVL2-2B, a model chosen because it differs on every axis that could confound: a 24 layer InternLM2 backbone, ordinary one-dimensional RoPE rather than M-RoPE, and 448 pixel dynamic tiling rather than a merged patch grid.

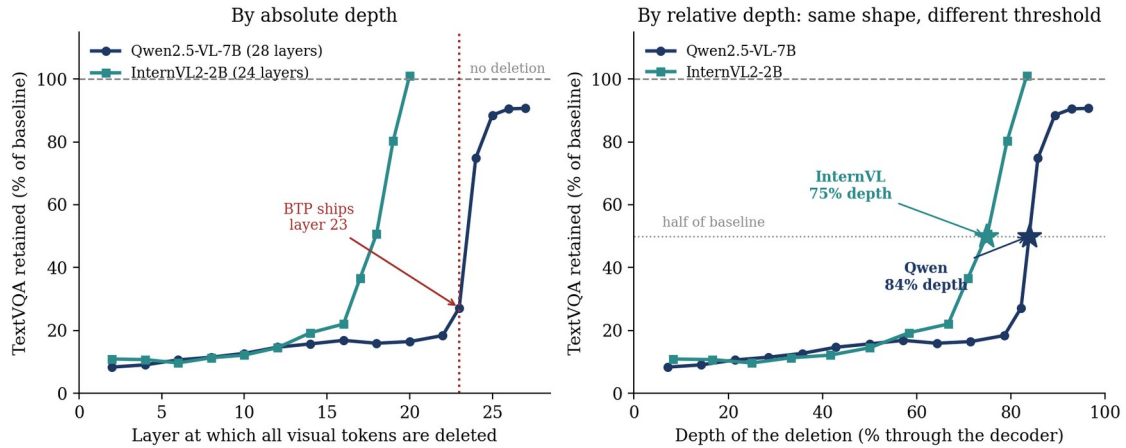


Figure 2. Left, by absolute layer. Right, by relative depth. Both models hold a flat floor near 10 to 18 per cent, then recover sharply.

Both curves have the same shape. Accuracy sits on a floor while the deletion happens early, then rises steeply once it happens late enough. The models keep drawing on image tokens until a certain depth, and after that they no longer need them.

Model	Layers	Half-of-baseline crossing	Relative depth
InternVL2-2B	24	layer 18.0	74.8 %
Qwen2.5-VL-7B	28	layer 23.5	83.9 %

Table 3. The depth at which each model has finished reading. Sharp, and architecture-specific.

6.1 What this costs BTP

BTP's released configuration deletes at layer 23 of Qwen's 28. Our sweep places Qwen's threshold at 23.5, so the shipped constant lands on the last layer at which total discard is still ruinous.

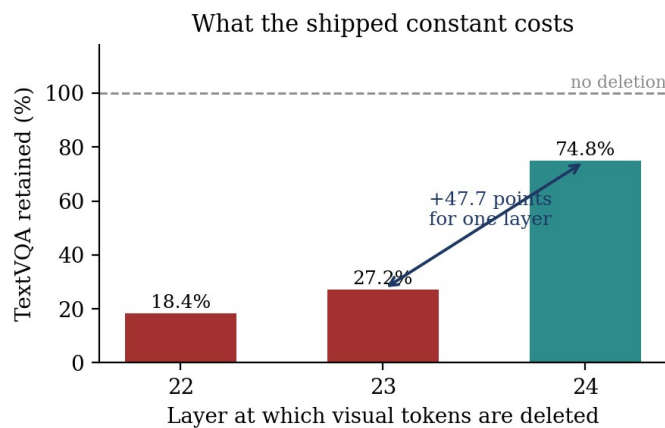


Figure 3. One layer either side of the threshold, measured on TextVQA at 500 samples.

Had the constant been 24 rather than 23, the same code would have scored about 0.645 instead of 0.234. This is not a subtle tuning question. It is a cliff, and the released value sits at the bottom of it.

6.2 One difference we did not expect

The two models do not behave identically at the top of the curve. InternVL recovers fully, reaching 101 per cent of baseline when the deletion is placed at layer 20 of 24, with four layers still to run. Qwen never quite does. It plateaus at 90.7 per cent even when the deletion is placed at layer 27 of 28. So Qwen continues to draw on visual tokens right to the end of the stack for roughly nine per cent of cases, while InternVL has genuinely finished with them. We report this because it was not predicted and it bears on how safe any late-stage discard can be.

7. Results Across the Suite

With the mechanism understood, we evaluated a corrected arm on the full benchmark suite. Baseline and as-released numbers were reused from earlier phases with matching sample limits, so only the corrected arm needed fresh GPU time.

Benchmark	Baseline	As released	Corrected	Kept
TextVQA	86.2	23.3	80.5	93.4 %
DocVQA (ANLS)	94.7	19.2	76.8	81.1 %
ChartQA	76.8	29.0	45.4	59.1 %
AI2D (control)	86.4	79.6	79.2	91.7 %
GQA	60.9	55.9	58.8	96.5 %
POPE (accuracy)	87.6	86.2	86.3	98.5 %
MME (perception)	1674.5	1658.7	1651.2	98.6 %
MMBench-EN	83.7	79.3	79.0	94.5 %
ScienceQA-Image	88.1	85.1	85.0	96.5 %

Table 4. Qwen2.5-VL-7B. Highlighted rows need the model to read.

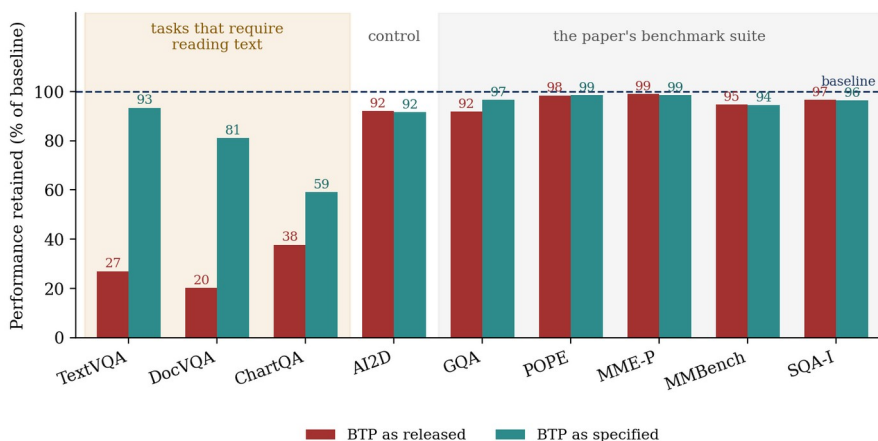


Figure 4. The two arms are indistinguishable everywhere except on the three reading tasks.

7.1 The benchmarks cannot see the fault

On POPE, MME, MMBench and ScienceQA the released arm and the corrected arm are the same within noise: 86.2 against 86.3, 1659 against 1651, 79.3 against 79.0, 85.1 against 85.0. Those tasks do not consult the image after layer 23.

Averaged across the paper's own five benchmarks, the released code retains 96.1 per cent of baseline against a claimed 96 to 98 per cent. The corrected arm gives 96.9 per cent. The fault does not merely slip past individual benchmarks; it slips past the average. Anyone evaluating on that suite would call the implementation a clean success. On the three reading tasks the same two arms score 28.4 and 77.9 per cent.

8. Pruning Has a Cost of Its Own

A one-line explanation is tempting and easy to overstate. Graded pruning to 12.5 per cent, with no deletion at all, still costs something, and the cost tracks how much reading the task needs. We measured this on InternVL2-2B, where we implemented the pruning ourselves and where no fault is present.

Benchmark	Baseline	Pruned to 12.5 %	Kept
TextVQA	0.716	0.468	65.4 %
DocVQA (ANLS)	0.835	0.303	36.3 %
ChartQA	0.586	0.398	67.9 %
AI2D (control)	0.746	0.728	97.6 %
GQA	0.592	0.546	92.2 %
POPE (accuracy)	0.888	0.848	95.5 %
MMBench-EN	83.4	81.0	97.1 %

Table 5. InternVL2-2B, graded pruning only, no deletion. Reading tasks lose roughly eight times as much as the standard suite reports.

The text suite averages 56.5 per cent of baseline while the standard suite averages 94.9 per cent. There is no bug here at all. The pruning is correct and behaves exactly as designed, and the benchmark suite still reports about 95 per cent while the model has lost close to half its reading ability.

The point

On Qwen2.5-VL there is a fault, and the suite cannot see it. On InternVL2-2B there is no fault, and the suite still cannot see what pruning costs. The blind spot is not a property of one bad implementation. It is a property of the evaluation protocol.

One caveat belongs with this table. Our InternVL selector implements the diversity component only; the attention component is omitted because InternLM2's FlashAttention-2 path does not expose attention scores. Attention is known to over-sample text, so these figures are an upper bound on what BTP itself would cost, not a like-for-like comparison. The direction and the asymmetry hold; the magnitude is inflated.

9. Limitations

- We cannot say what the authors intended, or which settings produced their published table. Our claim is about reproducibility, not about anyone being wrong.
- Two models is two models. The threshold is sharp in both and appears at a similar relative depth, but three points would make a trend and we have two.
- The threshold is measured on TextVQA alone. Whether it sits at the same depth for document or chart understanding is untested.

- The InternVL implementation is ours, not the authors'. It uses the diversity selector only, with layer numbers scaled across from Qwen rather than calibrated.
- The oracle experiment was run at one budget only, after an unexplained slowdown forced us to abandon the sweep.
- Runs use a 500 sample subset. Limits match across arms so comparisons hold, but they are not directly comparable to full-dataset figures published elsewhere.
- **Two claims withdrawn.** Phase 2 blamed the collapse on text having almost no redundancy; our own measurement refutes the premise, since text patches are less alike than background patches, 0.379 against 0.438. And an early result suggesting that surviving text coverage predicts a correct answer vanished at a larger sample. Neither should be quoted.

10. Conclusion

Visual token pruning methods decide, somewhere in their code, a layer at which the model no longer needs to look at the image. That decision is usually a constant. Our measurements show the quantity behind it is real, sharp, and different in each architecture: three quarters of the way through InternVL2-2B, and a little over four fifths through Qwen2.5-VL. Nobody measures it, and the standard benchmark suite cannot detect getting it wrong, because not one of its six tasks asks the model to read.

That is the useful lesson, and it is not about one line of code. Two practical recommendations follow.

- **Measure the threshold before choosing a pruning layer.** The sweep is cheap. Ours took fourteen short jobs per model and needed no training, no calibration set and no changes beyond one integer.
- **Report at least one text-reading benchmark.** Not because reading matters more than reasoning, but because it is the ability most sensitive to losing visual evidence, and therefore the best early warning. Methods should be measured where they are most likely to break.

10.1 Next steps

- Settle the framing with the mentor: implementation defect, or reproducibility finding.
- Extend the threshold measurement to DocVQA and ChartQA, and ideally to a third architecture.
- Correct the earlier report and deck, publish one tidy repository release, and begin the manuscript.

10.2 A note on method

Four experiments returned negative before the cause was found, and one hypothesis was refuted by a measurement we had predicted would confirm it. An earlier draft of this report concluded the effect was specific to Qwen, on the strength of a single measurement in InternVL; the sweep showed that to be wrong. We record this because the negatives are what ruled out the alternatives, and because the pattern they formed, four unrelated changes all landing on the same number, was itself the clue. A flat curve is not a disappointing result. It is a signature.

Every number here is traceable to a marked entry in the results log. Principal jobs: 411681, 412401, 414662, 416734, 416735, 416896 to 416904, 417968, 418974 to 419003, 419005 to 419012, 421205 to 421218, 421231 to 421237.